SDK, CLI y worker
Guía técnica

Keepseam mantiene integraciones desde el entorno del cliente: observa contratos, identifica usos potencialmente afectados, prepara propuestas y valida cambios autorizados en copias aisladas. El SDK y la CLI comparten implementación TypeScript.

> **Alcance entregado:** discovery JS/TS; selección explícita Python/Java; OpenAPI local/HTTPS, evidencia MCP/schema/documentación, propuestas con consentimiento, revisión semántica, pruebas aisladas y draft PR separadamente autorizado. El daemon no aprueba cambios. Calidad IA, publicación live y distribución firmada requieren aceptación específica.

```
Fuentes aprobadas → evidencia local → impacto JS/TS
  → propuesta → revisión humana → aprobación de validación
  → baseline/candidato en Docker → aprobación de publicación
  → GitHub draft PR → revisión/adopción verificadas por separado

Cloud ← únicamente estado permitido y autenticado
Modelo / GitHub ← contexto o cambios aprobados directamente
```

Límites de producto

El código, parches, prompts, schemas privados y logs permanecen locales; el cliente puede autorizar su contexto a un modelo o cambios a su GitHub. Esas rutas no pasan por Keepseam Cloud. Importar el SDK no inicia workers, abre DB, instala herramientas ni ejecuta código consumidor. La funcionalidad local no necesita cuenta Keepseam.

Componente	Contrato
SDK / CLI	`keepseam-client`, `/protocol`, `/cli`; CLI `keepseam`.
Evidencia / planificación	Snapshots y hashes, inspección acotada, versiones/procedencia, recetas exactas.
Worker / ejecución	SQLite privado, jobs y cola con recibos locales; Docker Linux x64 fijo.
Integraciones externas	Modelo, GitHub y Cloud tienen contratos y permisos independientes.

Un diff estructural no infiere semántica comercial. Un test verde solo demuestra sus aserciones. Una propuesta no equivale a autorización; un draft PR no acredita merge, adopción o despliegue. Las llamadas dinámicas, wrappers/SDK no cubiertos y formatos fuera del perfil permanecen explícitamente sin resolver.

Esta es la documentación técnica mantenida. Fuente: `docs/technical-guide.html`; PDF: `docs/technical-guide.pdf`. Las guías de fixtures y laboratorios conservan su alcance de ejemplo, separado del contrato de producto.

Distribución y SDK

El paquete privado `keepseam-client@0.1.0` es ESM, `UNLICENSED`. Requiere Node **24.21.0 ≤ versión < 25**; desarrollo usa pnpm **11.19.0**. No se ha autorizado publicar en npm. No instalar un paquete ajeno de nombre similar.

Desde el checkout

```
pnpm install --frozen-lockfile
pnpm engine:setup
pnpm build
pnpm keepseam --help
# Offline solo si las dependencias ya están en el store:
pnpm install --offline --frozen-lockfile
```

`engine:setup` descarga oasdiff **1.33.0** oficial, verifica checksum y conserva su licencia en `work/tools/oasdiff`; necesita `tar`. Es una adquisición explícita con red. Comparar contratos no descarga el motor automáticamente; se indica su ruta confiable y `.exe` en Windows.

Instalar el SDK en otro proyecto

```
pnpm pack --pack-destination work/packages
# En el proyecto receptor, usar la ruta real del archivo:
pnpm add --offline --ignore-scripts /ruta/keepseam-client-0.1.0.tgz
pnpm exec keepseam --help
```

Packing compila; no publica. El paquete SDK se instala con su runtime/dependencias y no incorpora las fixtures ni oasdiff. Offline exige cache local. Usar imports públicos, sin acoplar consumidores a `src/` o `dist/`; TypeScript usa ESM/NodeNext y tipos Node.

```
import { compareOpenApi } from 'keepseam-client';
const report = await compareOpenApi({
  basePath: '/contracts/base.json',
  revisionPath: '/contracts/revision.json',
  enginePath: '/opt/keepseam/engine/oasdiff',
});
console.log(report.status, report.changes);
```

Archivo CLI con runtime incluido

`pnpm release:build` produce un archivo por plataforma con ejecutable Node SEA, `lib/`, dependencias materializadas y motor/licencias. Extraer el directorio completo; no requiere Node instalado para la CLI. SDK, supervisor y host Docker conservan sus propios requisitos. El builder rechaza runtimes anteriores a 24.21.0, versiones 25+ y prereleases antes de crear artefactos; verifica help y una comparación real; no acredita publicación remota ni toda la instalación.

Windows no tiene firma Authenticode y macOS solo firma ad hoc, sin notarización. Verificar origen/digest y política de ejecución antes de instalar. Git para publicación debe aceptar `git --no-lazy-fetch --version`. Observación/propuesta funciona en Windows/macOS; validación consumidor exige Linux x64 y Docker local.

La CLI es `pnpm keepseam` desde checkout, `pnpm exec keepseam` desde paquete, o el ejecutable del bundle. Añadir `--silent` al script pnpm cuando se necesite JSON sin banners. Errores CLI se escriben en stderr; leer el contrato del comando para interpretar sus códigos.

Comparar e inspeccionar

```
keepseam compare /contracts/base.json /contracts/revision.json \
  --engine /opt/keepseam/engine/oasdiff --json
keepseam scan /projects/consumer --json
keepseam impact /contracts/base.json /contracts/revision.json \
  --consumer /projects/consumer --provider-base-url https://api.example.invalid \
  --engine /opt/keepseam/engine/oasdiff --json
```

Las rutas/dominio son placeholders. OpenAPI admite JSON 3.0.x/3.1.x y motor fijado. `compare` devuelve 0 sin breaking findings, 1 con cambios incompatibles y 2 con input/motor fallido. `impact` conserva breaking findings incluso si no encuentra un uso consumidor. Todos los informes de impacto requieren revisión.

Inspección sin ejecutar consumidores

`scanConsumer` y `analyzeImpact` capturan sintaxis TS/JS y manifests sin importar módulos, resolver imports, ejecutar configuración Babel, instalar dependencias ni lanzar scripts. La evidencia `fetch` registra ubicación, método/origen conocido y patrón de ruta; soporta segmentos completos `/orders/{id}`, no parciales como `{id}.json`.

Lecturas directas de JSON aportan evidencia acotada del parser. Aliases, URLs dinámicas, wrappers, SDK y flujos indirectos pueden quedar sin resolver. Un informe vacío no prueba ausencia de integraciones. Query values y credenciales URL se excluyen; los informes aún contienen metadata privada y no son eventos Cloud.

Límite	Comportamiento
Captura consumidor	500 archivos, 2.000 entradas, 1 MiB/archivo, 16 MiB total, profundidad 20.
Entradas omitidas	Ocultos, declaraciones, dependencias, builds y symlinks, con razones. Root symlink rechazado.
Manifest / scripts	Rangos declarados y nombres/hashes de scripts, sin ejecutarlos; no prueban versiones instaladas.

Catálogo de integración y dependencias

`integration-check manifest.json` separa versiones API, SDK, webhook, protocolo/servidor MCP y versiones efectivas; provenance y evidencia respaldan retirement/support. Fechas desconocidas son `null`, con explicación. Su cobertura es `listed-only`. `defineIntegrationBoundary` añade metadata a una función preservando identidad, `this`, resultados y errores; no valida schemas ni repara tráfico.

```
keepseam deps-scan /projects/consumer/package-lock.json --json
keepseam deps-audit /projects/consumer/package-lock.json \
  --advisories /private/advisories.json --json
```

Perfil npm lockfile v2/v3; no pnpm/yarn ni resolución online de rangos/advisories. Correlaciona versiones exactas del catálogo suministrado: unlisted no significa libre de vulnerabilidades. Límites: 4 MiB, profundidad 64, 5.000 entradas, 1.000 advisories y 20.000 correlaciones. No instala ni consulta registry.

HTTPS, MCP y schemas

`http-subscribe` registra OpenAPI; `source-http-subscribe` registra catálogos MCP normalizados, JSON Schema, ejemplos o documentación. Se observa la URL exacta tras opt-in `--http`; registrar es offline. Sin opt-in se devuelve `http-not-enabled` y no se presenta una captura fresca.

```
# Política de ejemplo ficticia, sin credenciales:
{"url":"https://api.example.invalid/openapi.json",
 "allowedOrigins":["https://api.example.invalid"],"contentKind":"json"}
keepseam http-subscribe /private/source-policy.json \
  --baseline /contracts/base.json --consumer /projects/consumer \
  --provider-base-url https://api.example.invalid \
  --engine /opt/keepseam/engine/oasdiff --state /private/keepseam
keepseam worker --once --state /private/keepseam \
  --engine /opt/keepseam/engine/oasdiff --http
```

HTTPS 443, allowlist exacta, DNS público validado y conexión fijada con TLS por nombre. Rechaza privados/reservados, redirects, compresión, query/fragment/credenciales URL. No usa proxy, cookies, crawl ni retries implícitos. Máximo 4 MiB/30 s, configurable a menos. El SDK permite credenciales por callback para un `authContextId` opaco; CLI/service solo perfil de fuentes públicas.

Preserva bytes, ETag/Last-Modified y contexto de autorización; 304 reutiliza solo snapshot matching previamente verificado. Captura/cache/comparación/historia se confirman conjuntamente bajo ownership. Un fallo conserva la última evidencia válida; no adelanta freshness. Cambiar alcance de permisos exige otro auth-context.

MCP y JSON Schema

```
keepseam mcp-compare /captures/before.json /captures/after.json --json
keepseam schema-compare /schemas/before.json /schemas/after.json \
  --direction input --json
keepseam source-subscribe /captures/after.json --kind mcp \
  --baseline /captures/before.json --state /private/keepseam
keepseam source-observe --once --state /private/keepseam --json
```

MCP exige captura completa, identidad server/protocol/auth-context, capabilities y pagination declaradas. Incompleto o contexto cambiado produce inconclusive; una ausencia no se interpreta como herramienta eliminada. No conecta servidores ni ejecuta tools. `mcp-impact --bindings` asocia receivers/ficheros explícitos a `callTool` y lecturas directas de `structuredContent`; los flujos restantes exigen revisión.

Schema soporta tipo único, propiedades/required/additionalProperties booleano, arrays/items y enum/const scalar. Rechaza refs, combinators, unions y constraints no soportados. Input preserva entradas antiguas; output exige que respuestas nuevas cumplan el contrato viejo. Límites: 4 MiB, profundidad 64, 20.000 nodos; MCP ≤ 1.000 tools. Números con pérdida de precisión son input no soportado.

`example-compare` y documentación solo muestran evidencia observada: no infieren required fields, variantes o compatibilidad. Texto/HTML no se renderiza ni ejecuta, no sigue links; normalizar saltos es opt-in. Esas fuentes no generan una migración autorizada por sí solas.

Proponer con un modelo

Seleccionar job observado, tests read-only y modelos explícitos. El cliente recomputa contratos/impacto y captura actual; un informe guardado no decide qué contexto revelar. No descarga ni descubre modelos por su cuenta.

```
# /private/local-model.json: manual por defecto
{"schemaVersion":1,"primary":"local","connections":[{
 "id":"local","revision":1,"credentialRevision":1,
 "provider":"local-openai-compatible","model":"your-installed-model",
 "policyId":"reviewed-v1","maximumOutputTokens":2048,
 "endpoint":"http://127.0.0.1:11434/v1/chat/completions"}]}
keepseam job-proposal-plan JOB_ID --state /private/keepseam \
  --engine /opt/keepseam/engine/oasdiff --model /private/local-model.json \
  --test orders.test.mjs
keepseam job-propose JOB_ID --state /private/keepseam \
  --engine /opt/keepseam/engine/oasdiff --model /private/local-model.json \
  --test orders.test.mjs --output proposal.json
```

`--source`: JS/TS, Python o Java capturado; `--evidence`: texto local revisado. Discovery automático sigue JS/TS; Python/Java conservan cobertura a revisar. Se envían sólo archivos seleccionados y contratos completos, sin truncar ni ampliar contexto.

Manual y Auto

Manual usa un destino; `"mode":"auto","fallback":"OTRO_ID"`, un principal y una alternativa. Terminal confirma destinos, modelos, revisiones, contexto y límites; remotos requieren `--allow-remote`. Consentimiento temporal de un uso; no autoriza tests/publicación. SDK rechaza getters/proxies/campos extra y captura opciones al iniciar.

`limits`: `timeoutMs` 1.000–120.000 (default 120.000), bytes input/output por intento (default 256 KiB; máximo 1 MiB), `maximumTotalOutputTokens` (default 8.192). El presupuesto reserva techos de generación y probes; no representa tokens facturados. Máximo dos generaciones; probes y verificaciones de evidencia consumen el mismo plazo global. `--timeout-ms` sustituye el plazo antes de consentir.

Credenciales, permisos, modelo o cuota inválidos bloquean esa conexión hasta una prueba explícita válida; Auto puede usar sólo la alternativa consentida y verificada, con aviso. Fallos transitorios degradan conexión. Cancelación, contexto/destino alterado, salida insegura y `decision-required` detienen la cadena. Una receta sigue pendiente de revisión semántica.

Comprobación funcional y claves

`model-connections verify --state DIR --model CONFIG` pide consentimiento para una inferencia sintética real, sin código privado: JSON fijo, máximo 128 tokens de salida y 30 s por probe. Se repite antes de trabajo si el recibo del proceso supera cinco minutos; nunca por polling inactivo. `status` no hace inferencia. Un reinicio exige nueva prueba; `--startup-checks` autoriza por separado esa comprobación del servicio.

El permiso de arranque se vincula a la configuración exacta: cambiarla exige verificar y consentir de nuevo. Sin permiso, el servicio indica unverified y no llama al modelo. Con permiso realiza una prueba por conexión seleccionada; informa healthy/degraded/requires-reconfiguration sin bloquear heartbeat.

Providers: OpenAI, Anthropic, Google o loopback HTTP explícito. Claves vía `apiKeyEnv` local; no se guardan ni pasan por Cloud. Cambiar configuración exige incrementar `revision`; rotar clave invalida recibos/consentimientos previos. Otro modelo declarado exige reconfiguración. Google acepta `ID` o `models/ID`: elimina sólo ese prefijo para la ruta y exige la misma versión en `modelVersion`, sin resolver alias (contrato REST). Loopback no demuestra aislamiento de red. Los tests controlados no acreditan calidad de IA real.

Revisar y autorizar

`job-propose` guarda con escritura exclusiva en `state/proposals/` y devuelve `artifactPath`, `proposalSha256` y plantilla de respuestas. Mantener la propuesta original inmutable y privada. Solo una propuesta ready con preguntas sin contestar se puede revisar.

```
# answers.json: copiar el digest exacto devuelto y todas las preguntas
{"proposalSha256":"SHA256_REAL_DEL_ARCHIVO_PROPUESTO",
 "answers":{"model-semantics":"Respuesta humana con evidencia"}}
keepseam job-review --state /private/keepseam \
  --engine /opt/keepseam/engine/oasdiff \
  --proposal /private/keepseam/proposals/proposal.json \
  --answers /private/answers.json --output reviewed-recipe.json
```

El JSON es orientativo: debe incluir **cada** ID obligatorio, incluyendo `keepseam-coverage` y preguntas de negocio cuando existan. No prellenar respuestas afirmativas. Cada respuesta es texto humano no vacío, ≤ 4 KiB; hash se refiere a los bytes exactos del archivo guardado, no a una serialización alternativa.

`job-review` valida digest, preguntas e identidad; vuelve a preparar validación y compara evidencia actual. Drift de consumidor, contratos o motor detiene el recorrido. Guarda otra receta, no altera propuesta ni otorga aprobación de tests/publicación.

Receta exacta y plan

Puede proporcionarse una receta propia revisada o la producida por el modelo. `plan`/`createMigrationPlan` comprueba hashes y reemplazos únicos sobre posiciones originales, sin solapamiento/cascada, traversal ni cambios en manifests. Un hash identifica contenido; no autentica a un aprobador.

Límites: receta 1 MiB, 50 archivos source, 100 edits/decisions. Pregunta sin respuesta produce requires-decision sin archivos. Ready-for-review sigue `authorization: not-granted`. JS sintácticamente válido no demuestra typecheck, equivalencia o permiso de ejecución.

```
keepseam job-plan JOB_ID --state /private/keepseam \
  --engine /opt/keepseam/engine/oasdiff \
  --recipe /private/keepseam/proposals/reviewed-recipe.json \
  --test orders.test.mjs --json
keepseam job-approve JOB_ID --state /private/keepseam \
  --engine /opt/keepseam/engine/oasdiff \
  --recipe /private/keepseam/proposals/reviewed-recipe.json \
  --test orders.test.mjs
```

`job-approve` presenta cambios, tests y recursos; requiere escribir el request ID exacto, sin `--yes` ni autorización piped. Un SDK caller explícito puede usar `approveJobValidation` bajo su propia política; no debe invocarlo automáticamente desde una respuesta modelo/Cloud no confiable.

El recibo se liga a estado, job, consumidor, contratos, receta, tests y perfil. Expira en 15 minutos y es de un uso. Cambios materiales exigen revisar de nuevo. Una negativa o pregunta de semántica no se elimina para hacer pasar tests.

Validar cambios

```
keepseam job-queue JOB_ID --state /private/keepseam \
  --engine /opt/keepseam/engine/oasdiff --recipe /private/reviewed.json \
  --test orders.test.mjs --approval RECEIPT_ID
keepseam run-approved --once --state /private/keepseam \
  --engine /opt/keepseam/engine/oasdiff --json
keepseam executions --state /private/keepseam --json
```

IDs/rutas deben corresponder al plan aprobado. Queue verifica vigencia y no consume aún el recibo; el plazo continúa mientras espera. Una receta/selección distinta requiere otro plan. Repetir admission con los mismos inputs vigentes es idempotente; solo intents queued son ejecutables.

Perfiles cerrados Linux x64

Exige `/usr/bin/docker`, daemon local `/var/run/docker.sock` e imagen del runtime adquirida explícitamente. Tests seleccionados determinan un único perfil; mezclar JS/TS, Python y Java se rechaza. Windows/macOS devuelve blocked; jamás ejecuta consumidor en host. Socket remoto/rootless y scripts de instalación están fuera del perfil.

```
# Adquirir expresamente la imagen, fuera de la aprobación:
docker pull node@sha256:51b1100cc2a83d370c6a60952e3f2989c8a43159d0e38586e090f3b3326efefd
```

Frontera	Límite aplicado
Snapshot	Source/manifests capturados; readonly. Sin checkout vivo, env, secretos, socket o dependencias del consumidor.
Tests	1–20 archivos de un runtime; receta no puede editarlos. Node --test, unittest/pytest o Java main/JUnit según perfil. Mismos bytes en ambas copias.
Contenedor	Sin red, pull=never, root readonly, usuario 65534, capabilities vacías, no-new-privileges/seccomp.
Recursos	Comunes: 1 CPU, 128 PIDs, stdout/stderr combinados ≤ 1 MiB y presupuesto adicional de limpieza de 5 s. Node/unittest/Java main: 512 MiB, tmpfs 64 MiB y plazo de 30 s. Pytest/Maven-JUnit: 768 MiB, tmpfs 256 MiB y 120 s. Plazo por ejecución, incluido preflight.

Sin manifiesto adicional: Python unittest o Java 25 `javac -proc:none`/main con assertions. Pytest y Maven/JUnit exigen preparar expresamente un runtime y pasar `--runtime` (pág. 16). Preparación descarga dependencias oficiales verificadas; ejecución del consumidor siempre sin red. Foreign preview indica `validation:not-run` hasta aprobar. Daemon/kernel y misma cuenta/administrador siguen dentro de la confianza; no sustituye CI completo.

Resultados conservados

`passed`: baseline/candidato pasan; `improved`: falla baseline y pasa candidato; `regression`: falla solo candidato; `failed`: ambos fallan. Infraestructura ausente, límites, cleanup no confirmado o ejecución incompleta produce blocked. Publicación solo admite candidato passing con evidencia durable y cleanup confirmado.

CLI ejecución devuelve 0 sin trabajo o todo passed, 1 con fallos completados/improvement y 2 blocked/interrupted/lost-ownership. Completed significa resultado registrado, incluso blocked. No modificar tests válidos ni borrar marcadores para reusar un recibo. El checkout original permanece intacto; reporte y output son privados.

Draft PR y reconciliación

Seleccionar antes de observar un repo consumidor independiente, committed, root exacto no-bare SHA-1 con `.git` local. Fixtures internas no cumplen este perfil. Preflight compara source/manifests capturados con index y blobs HEAD crudos en ambas direcciones; rechaza cambios no confirmados del perfil, worktrees, submodules, objetos externos o configuración peligrosa.

```
keepseam publication-plan INTENT_ID --state /private/keepseam \
  --git /usr/bin/git --engine /opt/keepseam/engine/oasdiff \
  --repository OWNER/REPO --base main --json > /private/candidate.json
keepseam publication-review --candidate /private/candidate.json \
  --author-name "YOUR NAME" --author-email "YOUR EMAIL" \
  --github --json > /private/publication-request.json
keepseam publication-approve --request /private/publication-request.json \
  --state /private/keepseam --consumer /projects/consumer
keepseam publication-publish --request /private/publication-request.json \
  --state /private/keepseam --consumer /projects/consumer \
  --approval PUBLICATION_RECEIPT_ID --github --json
```

Review solo lee GitHub y verifica repo privado, activo/writable/no-fork, immutable ID, permisos y base/tree exactos. `GH_TOKEN` se resuelve al request; preferir credencial corta y scope de un repo. La aprobación local presenta contenido/destino y exige request ID exacto; recibo propio 15 minutos/one-use, distinto del de tests.

El adapter crea blobs/tree/commit inmutables, una rama create-only `feat/api-…` y un draft PR same-repository GitHub.com. No sobrescribe ramas ni commits humanos, mergea o despliega. Revalida source y base; cambios de identidad/permiso/conflictos detienen writes. Races humanos pueden existir alrededor de crear PR; la comprobación no es un CAS atómico remoto.

Interrupciones y efectos inciertos

```
keepseam recover-approved --state /private/keepseam --json
keepseam publication-reconcile --request /private/publication-request.json \
  --state /private/keepseam --consumer /projects/consumer \
  --approval PUBLICATION_RECEIPT_ID --github --json
```

Execution conserva intención antes de create, ID antes de start y journal/outcome durable. Perfil, imagen, comando, tests y source quedan ligados a hashes. Los contenedores usan `keepseam-validation-<UUID>`. Recovery verifica ID, nombre y ownership con el perfil aprobado; rechaza journals con prefijos anteriores o ajenos, sin iniciar tests ni reencolar interrupted. Creación incierta conserva cleanup-unresolved incluso sin contenedor visible.

Publicación guarda intención durable antes de cada write. Timeout/respuesta perdida produce uncertain, sin retry automático. Reconcile es read-only: reconoce PR matching abierto/cerrado/merged y conserva cambios humanos; una PR ausente no prueba que el write no vaya a terminar. Mantener request/recibo/journal original. Simulation no acredita una PR real.

SQLite, archivos, Docker y GitHub no comparten transacción atómica ni garantía exactly-once. No mover/importar estado entre hosts como recuperación, borrar marcadores o editar DB. Un PR no cambia automáticamente baseline/adopción/deployment; esa reconciliación de contenido requiere aceptación propia.

Observación y estado

```
keepseam subscribe /contracts/revision.json --baseline /contracts/base.json \
  --consumer /projects/consumer --provider-base-url https://api.example.invalid \
  --engine /opt/keepseam/engine/oasdiff --state /private/keepseam
keepseam worker --once --state /private/keepseam \
  --engine /opt/keepseam/engine/oasdiff --json
keepseam subscriptions --state /private/keepseam --json
keepseam jobs --state /private/keepseam --json
```

Baseline/configuración se captura de forma inmutable; observar B no convierte B en versión adoptada. Repetir registro igual es idempotente. Jobs se deduplican por subscription, contratos, consumidor y motor; cambios de consumidor crean nueva evidencia, rollback conserva historia. Bytes cambiados pueden requerir review aunque no haya breaking findings.

El estado debe estar fuera del checkout y no contenerlo. Node SQLite usa transacciones/sincronización y leases con owner/generation; commit obsoleto es rechazado. Usar una versión LTS compatible y conservar backup privado. No usar un filesystem compartido/remoto ni editar filas para saltar permisos.

Almacén	Contenido y límites
`worker.sqlite3`	100 subscriptions, 1.000 jobs/transitions; contratos/reports ≤ 4 MiB, totales snapshots/reports 64 MiB cada uno.
`source-observations.sqlite3`	Catálogos/schemas/ejemplos/texto; 100 subscriptions, 1.000 transitions y 64 MiB de payloads.
Intents / attempts	1.000 intents, receta ≤ 1 MiB, ≤ 20 tests, 64 MiB recipes. Recibos/markers/outcomes privados y retenidos; SQLite v4 conserva runtime exacto.

Son límites de payload, no de tamaño exacto de DB. No hay pruning automático; agotamiento falla sin borrar evidencia o fingir freshness. Cambios de schema propios se migran transaccionalmente, conservando historia; DB ajena/futura falla. Ownership observer dura cinco minutos; ejecución diez y recovery cinco.

Conexiones de modelos: `model-connections.json` privado conserva hasta 32 identidades, revisión y diagnóstico, nunca claves o sus hashes. Un lock SQLite exclusivo impide escrituras concurrentes entre CLI/servicio. Estado anterior no concede una comprobación funcional fresca tras reiniciar.

Permisos y diagnóstico

POSIX: directorios 0700/archivos 0600. Windows: nuevas carpetas reciben DACL privada; paths existentes inapropiados fallan sin repair implícito. Rechazar symlinks/junctions, hardlinks runtime y directorios de red. La misma cuenta OS y administradores forman parte del trust boundary; modos/ACL no autentican un revisor humano.

Una observación fallida preserva último éxito/jobs y registra razón fija. `worker --once` devuelve 0 sin job actual pendiente, 1 con review y 2 busy/failure; unchanged no cierra jobs históricos. Plan/propuesta usan último snapshot capturado y no consultan proveedor silenciosamente; observar de nuevo antes de decidir y revisar timestamps.

`worker-loop` permite ciclos finitos y, solo con opt-in, ejecutar recibos ya aprobados. El servicio persistente observa, crea jobs y sincroniza metadata; jamás genera consentimiento, responde preguntas semánticas, ejecuta consumidor ni publica por sí solo.

Servicio y Linux

Versión revisada en ruta estable, cuenta no-root, consumidor/contratos readonly y estado writable. Registrar subscriptions antes de arrancar. `render` genera definición revisable; no instala servicios ni herramientas.

```
{"schemaVersion":1,"statePath":"/var/lib/keepseam/state",
 "allowHttp":false,
 "openApi":{"enginePath":"/opt/keepseam/bin/oasdiff","intervalMs":60000},
 "sources":{"intervalMs":900000},
 "models":{"configPath":"/etc/keepseam/models.json"}}
keepseam worker-service run --config /etc/keepseam/worker.json
keepseam worker-service status --config /etc/keepseam/worker.json
keepseam worker-service stop --config /etc/keepseam/worker.json
```

Quitar perfil sin registros; al menos uno es obligatorio. JSON ≤ 64 KiB, campos cerrados, rutas absolutas, sin credenciales/hooks. Intervalos 1.000–3.600.000 ms desde operación completada; cambiar config exige restart. Cloud: archivo privado, estado separado e IDs consentidos. `models` opcional exige permiso `verify --startup-checks` (pág. 5).

Observer serializa sus perfiles; Cloud tiene otro bucle serial para no retener heartbeat. Una observación OpenAPI nueva despierta sync; sin cambios, contacta cada 60 s como máximo. Fallos conservan backoff acotado y no permiten envíos paralelos. Una credencial rechazada invalida la licencia. El heartbeat no refresca evidencia histórica; medir aparte observación → Cloud → outbox → SMTP.

SQLite de servicio mantiene un lock exclusivo durante el proceso; crash lo libera. Status es snapshot durable, no prueba que PID siga vivo; consultar manager. Stop se liga al runId aleatorio, cancela HTTP y drena operaciones; un marker viejo no detiene el reemplazo. Manager permite 180 s antes de forzar cierre, no es watchdog de un proceso colgado.

Estado local inválido devuelve `INVALID_INPUT` sanitizado; stop no genera una orden con ese registro. Ausencia de estado devuelve `undefined`.

systemd system

```
keepseam worker-service render --config /etc/keepseam/worker.json \
  --target systemd --node /usr/bin/node --cli /opt/keepseam/installed-cli.js \
  --user keepseam > /private/keepseam.service
sudo systemd-analyze verify /private/keepseam.service
sudo install -m 0644 /private/keepseam.service /etc/systemd/system/keepseam.service
sudo systemctl daemon-reload
sudo systemctl enable --now keepseam.service
```

Resolver CLI instalado por `import.meta.resolve('keepseam-client/cli')` y usar su ruta real. Manager lanza Node directamente, sin shell/pnpm. System service exige usuario no-root, oculta homes, consumer/config readonly y state writable. Restart on-failure: 30 s, cinco starts/cinco minutos. Diagnóstico: `systemctl status`/`journalctl -u keepseam`; corregir causa antes de reset-failed.

La unidad incluye límites de recursos por servicio. Requisitos de cgroup v2, scope user y comprobación efectiva: pág. 22.

Logs locales sanitizados ≤ 1 MiB cada archivo actual/anterior, sin code/docs/URLs/keys/raw errors. Retención journal/OS es operación aparte. No usar el estado para coordinar dos hosts.

Managers y upgrades

macOS: launchd

Cuenta dedicada no-root, runtime/config/source readonly y estado privado writable. Para ejecución sin escritorio usar LaunchDaemon del sistema. Las rutas de ejemplo deben sustituirse por el Node/CLI público realmente instalado.

```
keepseam worker-service render --config /etc/keepseam/worker.json \
  --target launchd --node /usr/local/bin/node \
  --cli /opt/keepseam/installed-cli.js --user keepseam \
  > /private/com.keepseam.keepseam.plist
plutil -lint /private/com.keepseam.keepseam.plist
sudo install -o root -g wheel -m 0600 /private/com.keepseam.keepseam.plist \
  /Library/LaunchDaemons/com.keepseam.keepseam.plist
sudo launchctl bootstrap system /Library/LaunchDaemons/com.keepseam.keepseam.plist
sudo launchctl print system/com.keepseam.keepseam
```

RunAtLoad arranca; exit fallido reinicia con throttling. Stop/unload por `launchctl bootout system` y ruta plist. stdout/stderr del manager van a `/dev/null`; usar logs privados/print/ejecución foreground con la identidad prevista para diagnosticar arranque.

Windows: SCM y WinSW

Node no implementa service-entry-point Windows. El XML target es WinSW **2.12.0**, externo/MIT y adquirido/pinned/verificado por el operador. No se solicita escritorio. Identidad por defecto LocalService; otros servicios bajo la misma identidad comparten su autoridad. Para mayor separación, revisar host/cuenta dedicada.

Preparar DACL de estado y secretos con LocalService `S-1-5-19`, SYSTEM y Administrators únicamente, owner permitido y herencia equivalente. Runtime/config/source: read/execute; state: writable. No LocalSystem ni directorios de aplicación ampliamente writable. Las comprobaciones rechazan estado existente incorrecto sin arreglarlo silenciosamente.

```
keepseam worker-service render --config C:\ProgramData\Keepseam\worker.json \
  --target windows --node "C:\Program Files\nodejs\node.exe" \
  --cli "C:\Program Files\Keepseam\installed-cli.js"
# Guardar XML en UTF-8, junto al WinSW revisado: keepseam.exe/xml.
# Ejecutar install/start/status/stop/uninstall mediante el wrapper.
```

En PowerShell el ejemplo multilinea usa backtick o una sola línea, no backslash. SCM delayed auto-start; recovery 30 s/60 s/5 min. Stop solicita Ctrl+C/SIGINT y espera hasta 180 s; esperar SCM Stopped antes de cambiar archivos/start. XML omite stoparguments por el contrato de timeout WinSW 2.12.0. Wrapper logs rotan por separado.

Upgrade explícito

Parar manager, preservar DB, identidad, outbox/sequences, recibos y evidence, verificar bundle/motor, sustituir rutas fijadas y reiniciar. No downgrades de DB ni reset de approval markers. Rollback ejecutable solo si es compatible con estado nuevo; probar sobre copia. TUF staging no activa este proceso.

Validar arranque/reinicio/crash, permisos, logs y drenado en el manager real de cada plataforma. Renderizar una unidad/XML no es prueba de servicio instalado. La aprobación y ejecución consumidor permanece Linux x64 independientemente del supervisor.

Conectar a Cloud

Generar identidad local con `generateCloudWorkerIdentity()`; `privateJwk` permanece en custodia cliente. Entregar al owner/admin Cloud solo installationId/publicJwk/fingerprint. En panel workers se enrola tras passkey reciente, se compara fingerprint independientemente y se activa. El secret revelado una vez es reutilizable para token exchange hasta expiry/revocation; no es un token one-use.

Guardar `CloudCredentials` con privateJwk, secret y project wire identifier en archivo privado mediante secret manager. HTTPS obligatorio; `allowLocalHttp:true` solo loopback dev explícito. No hay comandos de enrollment en CLI producto; se usan API SDK + panel existente.

Selección y sync

Por subscription consentida, calcular `deriveCloudIdentifier(localId, installationId)` y asociarla a una integración enabled del mismo proyecto en Cloud. El ID no autoriza subir documentación o código. Sync requiere 1–100 IDs locales existentes, no UUID Cloud ni sus IDs derivados.

```
import { syncWorkerToCloud } from 'keepseam-client';
const result = await syncWorkerToCloud({
  statePath: '/private/keepseam',
  cloudStatePath: '/private/keepseam-cloud',
  credentials: privateCredentials,
  integrationIds: [approvedLocalSubscriptionId],
});
```

Service config añade `cloud:{credentialsPath,statePath,integrationIds,intervalMs}`, sin valores secretos inline; requiere perfil openApi. Outbox privado persiste eventos exactos, sequences e identidad de instalación. Lease renovable de 30 s excluye syncs simultáneos; ack perdido reutiliza mismo event ID/sequence/timestamp.

Frontera	Contrato
Protocolo v1	Allowlist ≤ 4 KiB: opaque IDs, enums, timestamps y counts. Rechaza campos extra, code/logs/prompts/schema/URLs/comandos.
Entrega	50 sends/invocación, ≤ 1.000 pending, sequences 0–1.000.000. Synced = outbox vacío; pending necesita siguiente sync; busy espera ownership.
Credenciales	Enrollment ≤ 90 días; token DPoP ≤ 5 min, emisión 10/min/installation, ≤ 50 tokens activos. Revocation comprobada en servidor.

DPoP P-256 autentica instalación y vincula token/prueba; no certifica la verdad de cada estado. Importar el encoder no abre conexión. El SDK manual continúa localmente; el daemon automático necesita licencia firmada vigente, incluso sin conexión. No clonar outbox a otro host ni borrar sequences/eventos para eludir rechazo; agotamiento exige preservación/enrolment nuevo.

Tiempo y privacidad

`recordedAt` mantiene creación job/update ejecución durante offline retries; heartbeat corresponde a sync actual. Ni heartbeat ni receipt refresca la observación del proveedor. Selected/passing counts cuentan test targets, no framework cases. Corregir reloj si futuras fechas superan dos minutos, sin reescribir evidencia.

Cloud muestra estado y correo attention opt-in; no aprueba ni ejecuta cambios. Proyección PR/adoption/Slack y autorizar ejecución remota requieren un contrato coordinado posterior; protocol enum disponible no prueba un feature conectado.

Updates verificados

`client-update` comprueba un repositorio TUF configurado y descarga paquetes inertes. Estado `downloaded-awaiting-approval`, instalación `not-authorized`. No extrae, ejecuta scripts/npm, cambia runtime, reinicia worker ni autoriza una migración. No hay root/feed público oficial incorporado ni update automático.

```
{"statePath":"/private/keepseam-updates","repository":{
 "metadataUrl":"https://releases.example.test/metadata/",
 "targetsUrl":"https://releases.example.test/targets/",
 "rootPath":"/private/release-root.json",
 "rootSha256":"SHA256_BOOTSTRAP_REVISADO"}}
keepseam client-update check --config /private/updates.json
keepseam client-update stage --config /private/updates.json
keepseam client-update status --config /private/updates.json
```

URLs/fingerprint son placeholders. Obtener root inicial y huella por bootstrap independiente del servidor de descarga; no confiar en ambos desde la misma URL no confiable. HTTPS público, paths distintos/no solapados terminados en slash, sin credenciales/custom ports/query/fragment/encoding. State propio privado, distinto del worker; config/root no writable por terceros.

Check refresca metadata sin paquete; stage descarga solo release más nueva; status valida archivo local sin red. Bytes ya verificados se reutilizan con recheck. CLI 0 completado/2 fallo; un timestamp guardado no acredita metadata remota actual. Cancelación no reporta éxito.

Contrato firmado

TUF vía `tuf-js@5.0.1`: firmas/thresholds, expiry, root rotation, rollback y target hash/length. Top-level target `channels/stable.json` contiene schemaVersion1, stable, x.y.z, sequence creciente, mínimo Node y artifact `packages/keepseam-client/x.y.z/keepseam-client-x.y.z.tgz` con hash/length firmados coincidentes. No delegated roles, prerelease/canales alternativos ni instrucciones remotas de instalación.

Reusar sequence para otros bytes, disminuir version/sequence o exigir Node incompatible falla. Solo 404 al siguiente root numbered termina rotation; otros errores/redirects fallan. Cambiar URL/huella en state existente es rechazado: rotar trust mediante TUF, no resetear DB. Clock/signing/expiry operados correctamente son requisitos; firma no demuestra código libre de fallos.

Boundaries y mantenimiento

SQLite exclusivo por operación; conserva root progress aunque falle red posterior. Paquetes `<sha256>.tgz` se flush/rename y revalidan. Inputs ajenos/dañados fallan, sin reset silencioso. Sin proxies, cookies o retry; DNS validado + pin TLS, rechaza compresión/redirect/body incompleto. Contenido se interpreta como datos.

Límites: paquete 32 MiB, root 512.000 B, timestamp/channel 16 KiB, otra metadata 1 MiB; 40 requests/40 MiB, headers 8 KiB, 16 root rotations y deadline compartido 120 s. Máximo tres paquetes completos y tres attempts interrumpidos; cache-full no borra nada. Revisar solo archivos updater propios sin operación activa; conservar trust DB. Directorio ≤ 64 entradas.

Un installer futuro requiere aprobación, drain, verificación de dependencias, schema-compatible rollback y health. Actualizar SDK es un cambio revisado del manifest/lockfile del consumidor; actualizar worker no prueba la migración API del cliente.

Especificación TUF · tuf-js.

Desarrollo y límites

Comando	Finalidad
`pnpm check`	Types, suite ordinaria con motor real y package consumer offline.
`pnpm test:isolated`	Integraciones Docker Linux x64 reales sobre perfil preparado.
`pnpm dev:local`	Compose persistente ficticio; seeds/editable provider separados de tests desechables.
`pnpm dev:apis`	Laboratorio documentación/SDK reales, consentimiento y helpers Cloud dev.
`pnpm dev:polyglot`	SDK instalado: recetas Python/Java revisadas y tests aislados; API ficticia.
`pnpm dev:model-evaluation` `pnpm dev:model-review`	Inferencia opt-in y revisión humana vinculada a hashes; fixtures sintéticas, no aprobación automática.

Evidencia necesaria para claims

La fixture de pagos usa sockets HTTP/TLS reales y relación explícita minor/major units, currencies, precision y fallos. Preserva tests consumidor: A pasa, B falla y la receta revisada debe mejorar call/parser sin alterar el modelo interno. Fixtures controladas y transporte modelo simulado no prueban una migración o inferencia live.

El piloto API live captura una transición real y declara schema parcial/recipe local: consultar `dev/live-validation/README.md` antes de autorizar. Una lista de diez docs no significa diez migraciones. Un modelo recomendado necesita pesos/runtime/digest/hardware exactos, memoria/tiempo, abstenciones, errores aceptados y esfuerzo humano. La evaluación actual no establece un perfil pequeño recomendado validado.

El loop generado exige consentimiento y revisión semántica; no acredita mantenimiento autónomo completo. Discovery Python/Java, SDK call graphs amplios, frameworks arbitrarios, auto-install/merge/deploy y proyección Slack/PR Cloud siguen pendientes. El feed TUF requiere claves y publicación operativa; no se crea confianza por disponer del builder. Incertidumbre de unidades o negocio detiene el cambio.

Licencias y dependencias

Source `UNLICENSED`: no hay autorización de distribución pública ni concesión Apache implícita. Runtime: Node built-ins, `@babel/parser@8.0.6` (MIT), `jose@6.2.12` (MIT), `tuf-js@5.0.1` (MIT); TypeScript Apache-2.0. oasdiff Apache-2.0 y licencias Node/transitivas acompañan su artefacto. Retener lockfile/notices y revisar advisories al actualizar.

Actualizar major/runtime de tuf-js exige revisar su compatibilidad y el boundary HTTP cubierto por regresión. Lifecycle scripts están deshabilitados por política del workspace.

Actualizar esta guía

```
# Editar docs/technical-guide.html y regenerar con renderer fijado:
node tooling/build-docs.mjs --playwright-root ../keepseam-cloud
node tooling/build-docs.mjs --check
# Renderer alternativo: work/doc-tools (@playwright/test@1.63.0 + Chromium)
```

Generador A4/Arial sin red, metadata fija y SHA256 fuente. CI verifica HTML/PDF; reproducción binaria exige navegador, sistema y fuentes idénticos. Actualizar guía y documentación de usuario Cloud con cada cambio de contrato antes de release.

Laboratorios controlados

Estos comandos prueban material ficticio desde el paquete público. Usar Node/pnpm fijados, motor provisionado y Docker Linux amd64; preparar imágenes y pesos explícitamente. No son perfiles de ejecución nuevos del SDK, instalaciones automáticas ni recomendaciones de modelo.

Un modelo local puede necesitar más de 30 s para cargarse en frío. Ante un fallo transitorio de conexión, la CLI indica revisar disponibilidad y preparar la carga en su servicio antes de verificar de nuevo. Keepseam conserva el máximo de 30 s por prueba y el plazo global de 120 s; no descarga ni precarga modelos automáticamente.

Inferencia CPU y revisión humana

Qwen2.5-Coder Instruct Q4_K_M	CPU / memoria	Pesos precargados
1.5B (por defecto)	4 CPU / 6 GiB	`work/model-evaluation/models/qwen2.5-coder-1.5b-instruct-q4_k_m.gguf`
7B (opt-in)	8 CPU / 8 GiB	`work/models/qwen-7b/qwen2.5-coder-7b-instruct-q4_k_m.gguf`

`dev/model-evaluation/profile.mjs` fija repositorio/revisión, bytes/SHA256 y digests llama.cpp/Node. Verifica el archivo entero; no descarga ni sustituye un modelo fallido. Qwen: Apache-2.0; llama.cpp: MIT. CPU sin GPU, una petición, semilla 42/temperatura 0; fixtures fijas no garantizan inferencia idéntica entre máquinas.

```
pnpm dev:model-evaluation
pnpm dev:model-evaluation --profile 7b
# Endpoint local para el flujo SDK/CLI, no servicio instalado:
node dev/model-evaluation/server.mjs start 43239 7b
node dev/model-evaluation/server.mjs stop RUTA_SERVER_DEVUELTA
```

Evaluación: contexto 4.096 tokens, salida 1.024 tokens/16 KiB y deadline 120 s. Servidor explícito: contexto 16.384, relay solo loopback y timeout 120 s; el llamador conserva su límite propio. Namespace modelo sin salida externa, pesos readonly, usuario no-root y límites de recursos. El modelo no monta checkout, secretos ni Docker socket.

Tres casos sintéticos, una inferencia por caso: cambiar endpoint/campo conservando el DTO; abstenerse ante unidades ambiguas; resistir instrucciones maliciosas en documentación. Examinar `work/model-evaluation/run-UUID/output/evaluation.json`: latencia, hashes, procedencia, memoria cgroup y rechazos. Timeout/rechazo conserva el fallo; solo `decision-required` esperado acredita abstención.

```
pnpm dev:model-review RUTA_RUN --prepare
# Completar revisión, decisiones y autorización solo de tests ficticios:
pnpm dev:model-review RUTA_RUN --answers REVISION.json
node --test dev/model-evaluation/classification.test.mjs
node --test dev/model-evaluation/semantic-review.test.mjs
```

La revisión separada vincula evaluación/propuesta exactas y todas las respuestas semánticas. Sin ellas: `awaiting-semantic-review`, sin ejecución. No sustituye recetas rechazadas. Baseline debe fallar y candidato pasar los mismos tests inalterados en copias aisladas; esto no autoriza consumidor live ni publicación, ni establece éxito general de un modelo real.

Evaluación de propuestas Python y Java

Generación con Codex

`pnpm dev:model-codex` usa la sesión existente de Codex, fixtures fijas y un SDK instalado sin red. Requiere `KEEPSEAM_CODEX_EXECUTABLE` absoluto y `KEEPSEAM_CODEX_MODEL` verificado. Una generación/caso, 120 s, herramientas desactivadas; misma revisión posterior. No acredita adaptadores BYOK ni capacidades del worker instalado.

`pnpm dev:model-codex --polyglot` selecciona seis casos fijos Python/pytest y Java/Maven-JUnit: migración monetaria, unidades ambiguas y documentación maliciosa en cada lenguaje. El modelo recibe únicamente adaptadores originales y tests de solo lectura; jamás el candidato manual de otro laboratorio. La revisión liga entrada, propuesta, preguntas y respuestas mediante hashes. El revisor puede registrar `rejections` con motivo sin ejecutar, sustituir ni publicar el parche. Un rechazo conserva el resultado fallido de la evaluación; no se convierte en abstención correcta.

Revisión y autorización de copias sintéticas

```
pnpm dev:model-review work/model-evaluation/run-UUID --prepare
# Linux x64: revisar el JSON y autorizar únicamente las copias sintéticas
pnpm dev:model-review work/model-evaluation/run-UUID --answers review.json \
  --engine work/tools/oasdiff/oasdiff --runtimes work/audit
```

El directorio de runtimes debe contener `pytest-runtime.json` y `maven-junit-runtime.json`, preparados mediante los comandos del perfil. El SDK instalado ejecuta prepare/approve/execute con baseline y candidato idénticos salvo el parche revisado. Mantiene los tests enviados al modelo y añade comprobaciones independientes de formato, Unicode, precisión y límites que no se enviaron a inferencia. Código original, tests y POM quedan intactos. Sólo un resultado SDK `improved`, baseline fallido, candidato aprobado y ambos contenedores retirados acredita mejora; exit code 0 con limpieza fallida sigue siendo fallo. La revisión semántica continúa siendo necesaria aunque pasen los tests.

Lenguajes y replay

Python y Java: selección explícita

```
docker pull eclipse-
temurin@sha256:51ae87bbb1949c1afdff51f6a3e6c5f283d84a4bd68512c9657fdc6605026a49
docker pull python@sha256:f040863673aea2570c3ff6a5c3fb4c673a016cbc5375005ad145915922b6b78a
pnpm dev:polyglot /absolute/work/keepseam-polyglot-unique
```

Linux x64, Python host 3.13 y salida nueva fuera del checkout. SDK instalado: captura → job → receta manual → aprobación → cola → baseline falla → candidato pasa. Source intacto, tests idénticos, cleanup verificado. Proveedor ficticio con sockets reales: precisión, moneda, unidades, límites, 429/503 y XXE Java.

Perfiles conservados: Python 3.13.16/unittest y Temurin 25.0.4.1+1/javac-main. Unittest exige ≥1 caso por archivo, rechaza cero casos/SystemExit y no ejecuta `__main__`; funciones pytest requieren el perfil siguiente. Java compila sin annotation processors y ejecuta con `-ea`.

Pytest y Maven/JUnit preparados

```
keepseam runtime prepare pytest --output /private/pytest.json
keepseam runtime prepare maven-junit --output /private/junit.json
# Añadir --runtime /private/pytest.json (o junit.json) en:
# validation-plan/approve/validate; job-plan/approve/queue/review
```

Preparación Linux x64 descarga artefactos oficiales con SHA verificado: pytest 9.1.1/Python 3.13 o Maven 3.9.16/JUnit 6.1.3/Java 25. SDK: `prepareValidationRuntime()`. Manifiesto liga imageId inmutable, receta y digest de dependencias al consentimiento y recuperación. El código consumidor jamás se ejecuta con red; no se aceptan tags, hooks ni comandos en el manifiesto.

Pytest desactiva autoload de plugins. Maven admite POM raíz, `src/test/java` y dependencias preparadas JUnit/compiler/surefire/resources; otros frameworks, dependencias y multimódulo quedan fuera. Tests, conftest, POM y manifiestos Python son inmutables. No ejecutar pip ni descargar dependencias del cliente durante validación.

El escáner informa `unsupported-language`/CLI 1: no deduce llamadas Python/Java. La propuesta exige source explícito; sintaxis foreign se valida tras autorización, nunca en preview. Tests `test_*.py`, `*_test.py` y `*Test.java` son readonly y participan en hashes; cambios de perfil requieren observar y aprobar de nuevo.

Las recetas del laboratorio son manuales: no prueban calidad IA ni soporte para todos los frameworks. La evaluación Qwen conserva su resultado parcial y abstenciones. Pruebas HTTP de proveedores controladas no acreditan inferencia real BYOK.

Grabación del terminal

```
pnpm build
pnpm engine:setup
node dev/terminal-demo/record.mjs
```

Terminal exige `pnpm build`, `pnpm engine:setup`, imagen Node fijada, Git `/usr/bin/git` y Docker `/usr/bin/docker`. Crea estado privado en `work/terminal-demo`, valida baseline/candidato con los mismos 25 tests y autorización de publicación separada, transporte GitHub simulado. Revisar `public-replay.json` antes de copiarlo a Cloud; conservar recibos privados. No implica un PR real ni métricas de usuarios reales.

Licencia y pausa segura

Todo `worker-service` automático necesita una licencia ES256 con propósito `keepseam-worker-license+jwt`. Sin ella arranca vivo pero **paused**, sin observar ni sincronizar eventos de negocio. El SDK y los comandos locales manuales no necesitan esta licencia. Pausar Keepseam no detiene procesos, APIs ni bases de datos del cliente.

```
{ "schemaVersion": 1, "statePath": "/var/lib/keepseam",
  "license": {
    "credentialsPath": "/etc/keepseam/credentials.json",
    "trustPath": "/etc/keepseam/license-trust.json",
    "leasePath": "/etc/keepseam/initial-license.json"
  }, "sources": {} }
```

Ejemplo parcial: registrar las subscriptions antes de iniciar. Credenciales y lease son archivos privados; el lease contiene una cadena JSON JWS, no un objeto inventado. Trust contiene `{schemaVersion:1,issuer,keys:[publicJwk]}`, sin claves privadas. Provisionar sus claves públicas por un canal independiente del endpoint que firma; no adoptar automáticamente un `kid` recibido.

Canal limitado de control

`createCloudControlTransport()` usa DPoP `worker:control`, separado de `worker:status`: sondeo cada 60 s, HTTP máximo 10 s, sin redirects. Una revisión o estado nuevo despierta el ACK serial, mínimo 1 s entre peticiones; un fallo espera el intervalo normal. Licencia vinculada a worker/org, installation/project, issuer y clave. Confirma `active`, `draining` o `paused`; solo cancela observación propia.

Una suspensión comercial conserva control y ACK, pero bloquea eventos de negocio. Reanudar exige ALLOW firmado nuevo y derechos vigentes. HTTP401 de identidad revocada invalida permiso offline inmediatamente. Fallos de red conservan únicamente un lease anterior válido; firma inválida, binding incorrecto o retroceso de revisión fallan cerrados. Workers anteriores al protocolo muestran `update-required`, nunca parada confirmada.

Tiempo y reinicios

ALLOW dura como máximo 86.400 s desde emisión y nunca más que entitlement/billing/credencial. No hay renovación por reiniciar ni por importar otra vez el archivo inicial. SQLite privado FULL conserva token, revisión, emisión, revocación y watermark de reloj. Un reloj anterior al watermark pausa; el host sigue siendo de confianza, no una barrera DRM ante su administrador.

Renovación automática mantiene derechos existentes; el token control no concede roles ni autorización de consumidor. Jobs, snapshots, recibos y outbox sobreviven pausa/reanudación. El `recordedAt` histórico no se reescribe: presencia y ACK miden contacto autenticado actual, no observación reciente del proveedor.

Rotación de autoridad

Cloud provisiona su keyring de licencia separado de auth, DPoP y TUF. La aprobación `worker-license.key.rotate` selecciona solo `kid` y SHA256 público ya configurados; no recibe rutas ni secretos. La selección/revisión se persiste. Distribuir confianza pública nueva antes de rotar y conservar la anterior hasta vencer leases previos de 24 h.

Firmar e importar versiones

`pnpm release:build` produce archivos portables, SHA256 y recibo `*.tar.gz.signing.json` ligado al archivo y ejecutable exactos. Linux necesita TUF. Windows sin certificado queda unsigned; macOS ad-hoc queda sin publisher/notarización. Un hash, certificado configurado o borrador GitHub no constituye una firma de editor válida.

Windows configura certificado de su store, SignTool absoluto, timestamp HTTPS y publisher esperado. Exige Authenticode Valid y subject exacto. macOS configurado firma Developer ID y somete el bundle a notarytool; exige Accepted y evaluación Notarized Developer ID. Sin certificados no hay recibo de editor verificado. Procedimiento y límites en la página siguiente.

Feed TUF local y paquete de importación

```
pnpm build
pnpm release:sign --config /absolute/operator/release.json
```

Configuración privada de operador: `schemaVersion:1`, rootFile/rootSha256, version, sequence creciente, sourceRevision Git40hex, notes/requirements ES/EN, documentationFile PDF y artifacts. Cada artifact fija os/arch/fileName/file y, para nativos, signingReceiptFile. RoleKeys contiene targets/snapshot/timestamp, cada uno con arrays `{keyid,file}`: archivos PEM privados Ed25519 que coinciden con las identidades y umbrales del root fijado.

Tras una renovación TUF, la siguiente versión debe fijar `metadataVersions:{targets,snapshot,timestamp}` superiores a los últimos publicados; la secuencia del producto es independiente. Procedimiento en pág. 21.

El comando escribe un directorio nuevo `work/releases/signed-VERSION-UUID`: metadata root/targets/snapshot/timestamp; targets de SDK, workers, canal stable, manifiesto y PDF; `import-bundle.json`. No publica ni instala. Root offline se provisiona fuera del comando. Claves de roles no aparecen en bundle/logs. Publicar los mismos bytes del feed es una acción operativa independiente.

Manifest firma versión, secuencia, commit, texto, archivo/plataforma, SHA/length y recibos nativos. El PDF canónico `keepseam-client-VERSION-technical-guide.pdf` también es target TUF. El updater selecciona el SDK ≤32 MiB sin instalarlo. Solo el adaptador Cloud usa tokens `worker:updates` en sus rutas autorizadas; nunca los envía a feeds externos.

Cloud: importación, aprobación y descarga

Provisionar directorio de artefactos readonly y root TUF exacto mediante `KEEPSEAM_TUF_ROOT_FILE` y `KEEPSEAM_TUF_ROOT_SHA256`. POST `/api/v1/platform/releases/import` con `{requestId,bundle}` y scope releases.manage requiere FIDO reciente. Bundle ≤30.000 B. Cloud ejecuta tuf-js real sobre inputs en memoria, sin URL externa, root rotation ni extracción/ejecución.

Se crea draft tras verificar firmas, expiración, targets y archivos. `client-release.publish/withdraw` y `client-advisory.publish/withdraw` usan aprobación separada y revisión/hash exactos. Publicar revalida metadata, bytes/PDF, publisher configurado y secuencia superior; artefactos nativos unsigned permanecen draft. Editar contenido firmado exige otra importación/versionado, no una firma ficticia.

Solo cuentas con correo verificado descargan releases publicados y PDF correspondiente. Archivo se valida otra vez por SHA/length, responde attachment/no-store y navegador comprueba SHA antes de guardar. Withdrawal impide nuevas descargas; no borra ni revoca automáticamente copias ya distribuidas. Hasta provisionar claves/certificados y publicar una versión aprobada, el catálogo puede seguir vacío.

Del build a la instalación

Paquete para el operador Cloud

`release:sign` también genera `delivery/`. `cloud-artifacts/` contiene los archivos planos y el PDF; `trust/root.json` la raíz pública; `native-receipts/` los recibos disponibles; `import-bundle.json` incluye requestId estable y bundle. `delivery.json` fija versión, commit, secuencia y hashes de esos bytes. No contiene claves privadas ni publica.

```
KEEPSEAM_RELEASE_DIRECTORY=/operator/delivery/cloud-artifacts
KEEPSEAM_TUF_ROOT_FILE=/operator/delivery/trust/root.json
KEEPSEAM_TUF_ROOT_SHA256=HUELLA_APROBADA_POR_OTRO_CANAL
```

Provisionar archivos readonly para API; comprobar raíz y huella por un canal independiente antes de arrancar. Importar el JSON en Platform con releases.manage y FIDO reciente, comprobar procedencia/firmas y solicitar publicación. Otra identidad autorizada aprueba el digest. La cuenta cliente con correo verificado descarga archivo/PDF y comprueba SHA256. Descargar no acredita instalación.

macOS con identidad real

El builder acepta `KEEPSEAM_APPLE_SIGN_IDENTITY` (SHA1 de certificado local), `KEEPSEAM_APPLE_NOTARY_PROFILE` (perfil Keychain ya provisionado) y `KEEPSEAM_APPLE_PUBLISHER` (Developer ID Application exacto). Firma cliente y motor con hardened runtime y timestamp; concede al cliente permisos V8 JIT/memoria ejecutable. No exporta contraseñas ni aprovisiona cuentas.

Ditto prepara un ZIP; una llamada notarytool submit/wait tiene límite de 12 minutos. Exige Accepted, log sin errores y codesign/spctl de ambos ejecutables. Fallo no produce un archivo verificadamente firmado. Apple no permite staple en ejecutables planos/ZIP: la primera comprobación Gatekeeper necesita conectividad. Sin identidad solo queda ad hoc. Certificados reales requieren aceptación en macOS.

Versiones y CI

El tag coincide con package.json y pertenece al main revisado con SDK acceptance. El PDF identifica client-version y SHA de su fuente. Dispatch `sign_delivery:true` selecciona runners privados preprovisionados y entorno protegido `client-release-signing`; el modo normal construye borradores. Certificado/Keychain/TUF y token de docs se configuran antes: no se obtienen de clientes ni de una PR.

El runner TUF fija `KEEPSEAM_RELEASE_SIGNING_CONFIG` privado. El job enlaza sourceRevision con el SHA revisado, usa archivos descargados mediante KEEPSEAM_RELEASE_INPUT_DIRECTORY y adjunta delivery al borrador GitHub. No publica automáticamente GitHub ni Cloud. Si caduca metadata, renovarla por el procedimiento de pág. 21.

Instalar el archivo descargado

```
tar -xzf keepseam-0.1.0-linux-x64.tar.gz
./keepseam-0.1.0-linux-x64/keepseam --help
# Windows: extraer todo y ejecutar .\keepseam.exe --help
```

Preservar lib, engine, licencias y PDF. Comparar contratos manualmente funciona sin Cloud; worker-service exige enrollment, credencial DPoP propia y trust/licencia válidos. No instalar servicios SCM desde una prueba ordinaria. Una clave efímera TEST demuestra ese laboratorio, no un editor comercial ni certificados reales.

Renovar metadatos TUF

El manifiesto, sus archivos, PDF, canal, commit y raíz permanecen byte a byte. Solo se renuevan metadatos de vigencia y sus firmas. Este procedimiento no reconstruye código, cambia confianza, instala ni publica; las claves privadas permanecen en el entorno del firmante.

Preparar la renovación

```
pnpm release:refresh --config /absolute/operator/refresh.json
```

Configuración cerrada: `schemaVersion:1`, `releaseDirectory` absoluto del último directorio firmado completo, `rootSha256`, `manifestSha256` aprobados por otro canal y `roleKeys`. Solo necesita las claves de roles que renueva, siempre timestamp; no abre claves offline de targets/snapshot cuando conserva sus bytes. El directorio contiene `metadata/`, `targets/`, `import-bundle.json` y `delivery/`; conservarlo completo. La entrega plana no contiene los bytes del canal.

El comando autentica firmas y referencias anteriores aunque hayan caducado, comprueba todos los targets y rechaza cambios de manifiesto, archivo o raíz. La raíz debe seguir vigente: su caducidad requiere un procedimiento separado de rotación de confianza. Leer metadatos vencidos para volver a firmarlos no autoriza a clientes a aceptarlos.

Rol	Renovación
timestamp	Siempre versión anterior +1; vigencia máxima de una hora.
snapshot	Conserva bytes si quedan ≥1 h y targets no cambia. En otro caso +1, máximo un día.
targets	Conserva bytes si quedan ≥1 h. En otro caso +1, máximo siete días; descriptores idénticos.

Ningún vencimiento supera el de la raíz. El resultado es otro directorio privado `refreshed-VERSION-UUID` con nueva entrega; conserva la anterior como evidencia. La salida informa `metadataVersions`. Usar siempre la renovación más reciente: partir de una anterior puede producir un rollback que Cloud y clientes rechazarán.

Actualizar un borrador en Cloud

POST `/api/v1/platform/releases/:id/metadata` con `{requestId,expectedRevision,bundle}`, FIDO reciente y permiso de releases. Reutilizar el requestId del paquete para reintentos. Solo un draft con identidad y revisión exactas admite la renovación; se audita y cambia su revisión. Revisar y solicitar nuevamente publicación sobre esa revisión. Cloud no recibe claves ni renueva firmas.

Publicar la siguiente versión del producto

`release:sign` admite `metadataVersions:{targets,snapshot,timestamp}`, enteros 1–2147483646. Para la siguiente versión elegir los tres mayores que los últimos del feed, incluidos sus refresh. El valor por defecto usa sequence y solo basta cuando supera los metadatos existentes; después de renovar no se puede asumir. El cliente conserva su cache y rechaza retrocesos aunque aumente la versión del producto.

Publicar primero targets/snapshot y por último timestamp en el feed operado. Nunca sustituir bytes manteniendo su versión, extender artificialmente el TTL ni borrar la cache del cliente para eludir un rechazo. Contrato de actualización TUF §5.4.

Recursos y scope user

Cada servicio ejecuta su worker en un proceso propio. En Linux, la unidad generada configura estos límites para su cgroup y descendientes; los contenedores de validación Docker tienen sus propios límites independientes.

Directiva	Efecto solicitado al manager
`CPUQuota=100%`	Como máximo el equivalente a un núcleo de CPU.
`MemoryHigh=384M`	Umbral de presión de memoria; no es el límite duro.
`MemoryMax=512M` `MemorySwapMax=0`	Límite duro de memoria y sin swap para este servicio.
`TasksMax=128`	Límite de tareas, incluidos procesos e hilos.
`OOMPolicy=stop` `KillMode=control-group`	Detener el servicio tras OOM y cerrar sus descendientes al detenerlo.

Se recomienda **system scope**, con usuario dedicado no-root. cgroup v2 y sus controladores deben estar disponibles. El operador puede ajustar valores mediante un drop-in revisado tras medir su carga; el SDK no cambia los límites del host ni configura delegación automáticamente.

Servicio de usuario

Render con `--scope user`, sin `--user`; instalar en `~/.config/systemd/user/` y ejecutar `systemctl --user daemon-reload` y `systemctl --user enable --now keepseam.service`. Depende del user manager; no configura linger ni WSL.

Los controladores cpu, memory y pids deben estar delegados al user manager. systemd 249, utilizado por Ubuntu 22.04, requiere habilitar expresamente la delegación de CPU desde el sistema. Una directiva renderizada no demuestra que el kernel aplique su límite. Si falta delegación, usar system scope o configurar la política del host con su administrador.

El aislamiento usa namespaces, PrivateUsers, montajes readonly y tmpfs. Si el host rechaza el arranque con `226/NAMESPACE`, resolver su política o usar system scope; no retirar protecciones para obtener un arranque aparente.

Comprobar la instalación real

Consultar `systemctl show keepseam.service -p MainPID -p ControlGroup`, comprobar la pertenencia del proceso en `/proc/PID/cgroup` y leer `cpu.max`, `memory.high`, `memory.max`, `memory.swap.max` y `pids.max` bajo el cgroup correspondiente. Para scope user añadir `--user`.

Probar cierre, caída y reinicio con el manager del host. El presupuesto nativo de cierre sigue siendo 180 s; los ensayos controlados pueden usar plazos propios. Un proceso separado evita compartir una caída de Node, pero el host conserva recursos comunes. Windows y macOS tienen procesos separados; esta configuración no acredita límites duros equivalentes en esas plataformas.

Snapshots consistentes

Para una nueva raíz, la ceremonia offline debe fijar `consistent_snapshot:true` antes de firmarla y distribuir su huella por un canal independiente. El firmante acepta esa raíz exacta; no crea claves, cambia el flag ni renueva confianza. Las raíces existentes con `false` siguen verificándose y conservan sus rutas anteriores.

Separar custodia y transporte

`release:sign` y `release:refresh` devuelven `repositoryDirectory`. Solo su subárbol `repository/` es la raíz de transporte. Los directorios hermanos `metadata/`, `targets/`, el bundle y `delivery/` permanecen en custodia del operador para renovación/importación; no publicar el directorio firmado completo.

Transporte con raíz true	Vínculo verificado
`metadata/N.root.json`	Solo la raíz suministrada; no genera una cadena de rotación. Bootstrap mediante su huella acordada.
`metadata/N.targets.json` `metadata/M.snapshot.json`	Versiones independientes. Las referencias firmadas conservan los nombres lógicos targets.json y snapshot.json.
`metadata/timestamp.json`	Puntero de vigencia; referencia versión, hash y longitud del snapshot.
`targets/ruta/SHA256.nombre`	Prefijo hash sobre el nombre del archivo; incluye canal, manifiesto, SDK, portable y PDF.

Con true no se exportan aliases mutables de targets/snapshot ni archivos de destino sin prefijo. Con false se exportan sus nombres originales. Cada salida es un directorio nuevo: ninguna generación existente se sobrescribe. Sus archivos privados canónicos se conservan para el procedimiento de renovación.

Publicación y renovación

Un operador de feed debe conservar las rutas inmutables de generaciones aún en uso, instalar primero destinos y metadatos versionados y cambiar timestamp al final. Reutilizar una misma ruta versionada con bytes distintos es un conflicto, nunca una sustitución. El verificador rechaza mezcla de generaciones, hashes alterados y retrocesos; esta estructura también evita fallos por leer mitad de dos publicaciones legítimas.

La renovación offline puede volver a firmar targets si su vigencia lo exige; necesita su clave autorizada y mantiene todos los descriptores y bytes del producto. Cloud dispone de un proceso separado de renovación online para timestamp/snapshot, sin autoridad para cambiar targets o root. Requiere habilitar el feed y configurar la custodia de sus claves; los comandos Client no instalan ese servicio ni distribuyen claves.

Alcance actual

El actualizador verifica el SDK `.tgz`, limitado a 32 MiB, y lo deja pendiente de aprobación local. Cloud ofrece descarga manual y transporte autenticado para workers cuando se configura el feed con una generación vigente aprobada. La exportación local no publica una release; la publicación en producción y la renovación continua requieren operación. No instala automáticamente el portable.

Especificación TUF §6.2: nombres versionados y prefijos hash; las firmas, límites de descarga, caducidad y caché persistente siguen siendo obligatorios.

Updates desde Cloud

Con el feed habilitado y una release aprobada, el cliente consulta y prepara el SDK con la identidad de un worker. No descubre el root de confianza, instala paquetes ni programa comprobaciones. Los portables conservan su descarga independiente.

```
{"statePath":"/private/keepseam-updates","repository":{
 "kind":"cloud",
 "baseUrl":"https://cloud.example.test",
 "credentialsPath":"/private/worker-credentials.json",
 "rootPath":"/private/release-root.json",
 "rootSha256":"SHA256_BOOTSTRAP_REVISADO"}}
keepseam client-update check --config /private/updates.json
keepseam client-update stage --config /private/updates.json
```

URLs y huella son ejemplos. El origen HTTPS canónico debe coincidir con `baseUrl` del archivo privado de credenciales del worker; no admite HTTP local, puertos personalizados, path, query ni fragmento. El root inicial revisado requiere `consistent_snapshot: true`. Las credenciales conservan la clave privada P-256 local y deben tener permisos privados.

Autorización y transporte

El cliente solicita un token de cinco minutos como máximo con scope `worker:updates`. Cada GET firma DPoP con método, URL exacta, identificador nuevo y hash del token. Los tokens de estado o control no sirven para descargar; el token de updates permanece en memoria y no se usa para licencias. No hay cookies, headers de autorización configurables, redirects ni secretos en URLs.

Autenticación y descargas comparten el límite de 40 peticiones, 40 MiB recibidos y 120 s. El SDK sigue limitado a 32 MiB. Cada conexión valida todas las respuestas DNS, fija una dirección pública y exige TLS; una renovación de token conserva los límites originales. No se reintenta automáticamente una petición rechazada.

Resolver un bloqueo

Reason	Acción del operador
`credentials-unavailable` `authorization-rejected`	Revisar archivo privado, origen y acceso del worker; reconectar si procede.
`rate-limited` `network-unavailable`	Esperar o reparar conectividad/disponibilidad antes de una nueva comprobación.

`trust-expired`
`targets-expired`
`metadata-expired`
	Renovar y publicar los roles firmados afectados; conservar el historial de confianza.

`feed-unavailable`
`target-unavailable`
`release-withdrawn`
	Publicar una release aprobada disponible o restaurar sus bytes exactos; no sustituirlos por otro target.

Los errores conservan el root y las versiones verificadas. No borres la caché para eludir rollback o caducidad. `blocked` no autoriza instalación; `status` solo consulta estado local.